

Taste Matcher v3.1

Project Report

Project: Taste Matcher

Author: Srirama Prabhav R. — Developer & QA Lead

Date: May 4, 2026

1. What Is Taste Matcher?

Taste Matcher is a backend-first, locally-run Node.js application that solves a problem every serious film enthusiast knows: a Letterboxd watchlist grows into hundreds — sometimes thousands — of titles, and there is no native way to surface what you should actually watch next based on your personal taste.

The application reads your Letterboxd ratings history and your watchlist, enriches every title with metadata from the TMDb API, and runs a multi-dimensional mathematical scoring engine that predicts, for every unwatched film on your list, how much you are likely to enjoy it. The output is a ranked watchlist — sorted from your most likely next favourite to the least — displayed through a clean interactive UI and exportable as a Letterboxd-compatible CSV.

There is no cloud service, no paid subscription, no account, and no database server. Everything runs on your machine, persists via local JSON flat-files, and can be started with a single command.

Core Promise: *Tell Taste Matcher what you have loved. It will tell you what to watch next.*

2. Why It Is Needed

Letterboxd is the social network of record for film enthusiasts, yet its recommendation and sorting features are intentionally minimal. You can sort your watchlist alphabetically, by year, by average TMDb rating, or by date added — none of these dimensions capture personal preference.

Existing solutions fall into two camps:

- Streaming recommendation algorithms (Netflix-style) require massive datasets, user accounts, and treat taste as a black box. They cannot operate on a personal watchlist you have already curated.
- Generic watchlist managers sort by popularity or release date — neither of which is personal.

Taste Matcher occupies a unique middle ground: a personal, offline, mathematically transparent recommendation engine built specifically for the Letterboxd ecosystem. Every weight, every score component, every tie-break is visible and explicable. No black box, no subscriptions.

3. How It Works — The Full Pipeline

Stage 1 — Data Ingestion

The user places ratings.csv and watchlist.csv into the /data/ directory. The backend parses both files using csv-parse with automatic header detection that handles both legacy Letterboxd formats and the current v7 format. During parsing, the system performs overlap detection — identifying films present in

both files. These are flagged as rewatches and separated out: they contribute to taste model training but are excluded from ranked recommendations.

Overlap detection uses a dual-strategy approach: primary matching by Letterboxd URL, with a title + release year fallback.

Stage 2 — TMDb Metadata Enrichment

For every unique film across both lists, the backend calls the TMDb REST API to retrieve: genres, director and writer credits, production countries (mapped to regions), release decade, thematic keywords, franchise/collection membership, and the TMDb community vote average.

All responses are written to `tmdb_cache.json`. A film fetched once is never fetched again — keeping the application within TMDb's free-tier rate limits even for large libraries.

Stage 3 — Taste Profile Construction

The scoring engine builds nine distinct taste profiles from the complete ratings history, each Bayesian-smoothed to prevent outlier distortion (SMOOTH_K = 5):

Profile Dimension	What It Tracks	Example
Genre Affinity	Average rating per TMDb genre	Thriller: 4.1 Drama: 3.7
Keyword Affinity	Mood / theme / tone preferences	Surrealism: 4.6 Action: 2.9
Director Affinity	Per-director weighted average rating	Kubrick: 4.8 Bay: 2.1
Writer Affinity	Per-screenwriter weighted average	Kaufman: 4.7
Geography Affinity	Country and regional cinema preferences	Japan: 4.5 Asia: 4.2
Decade Affinity	Era-based taste	1970s: 4.3 2010s: 3.6
Collection Affinity	Franchise / saga preferences	Before Trilogy: 4.9
TMDb Score Norm	Community quality baseline	Normalized 0–1
Neighbour Similarity	k-NN over rated films	Top-30 neighbours, Jaccard similarity

Stage 4 — Nearest-Neighbour Similarity Layer

For each watchlist candidate, the engine computes a k-NN similarity score by scanning the entire rated history and computing Jaccard similarity across five feature sets: genres, keywords, production region, directors, and era. The top 30 neighbours above a 0.12 similarity threshold are retained.

```
filmSim = 0.50×mood + 0.20×regionJaccard + 0.15×eraScore + 0.15×directorMatch
moodSim = 0.65×genreJaccard + 0.35×keywordJaccard
neighbourScore =  $\Sigma(\text{sim} \times \text{rating}) / \Sigma(\text{sim}) \times \text{confidence}$  [confidence saturates at 8+ neighbours]
```

This layer distinguishes Taste Matcher from a simple genre-average calculator — a Japanese slow-burn psychological thriller will correctly identify Korean crime dramas as close neighbours, capturing regional and tonal similarity beyond broad genre tags.

Stage 5 — Final Scoring & Ranking

Each watchlist film receives a composite predicted affinity score (0.00–1.00):

Score Component	Default Weight	Rationale
Keyword Affinity	22%	Strongest proxy for tone & theme taste
Genre Affinity	18%	Foundational preference dimension
TMDb Community Score	15%	Quality baseline from aggregate ratings
Director Affinity	10%	Auteur recognition
Neighbour Similarity (k-NN)	10%	Structural similarity to loved films
Writer Affinity	8%	Screenwriting voice recognition
Country / Region Affinity	7%	Cinema culture preferences
Decade Affinity	5%	Era taste preferences
Collection Affinity	5%	Franchise / saga continuity

Weights sum to 1.00 (100%) and are fully user-configurable via the priority ranking system introduced in v3.1.

Stage 6 — UI Rendering & Persistence

The scored and ranked data is served via an Express REST API to a vanilla HTML/CSS/JavaScript frontend — no React, no Vue, no build step. All computed state is written to four flat JSON files in `/cache/`, ensuring zero computation overhead on subsequent launches.

4. A Complete Example — From CSV to Ranked Watchlist

A cinephile with 400 rated films on Letterboxd. Highest-rated films cluster around Japanese cinema, 1970s European arthouse, psychological thrillers, and character-driven dramas. Watchlist contains 230 films.

Step	What Happens
1 — Export & Setup	User exports ratings.csv and watchlist.csv, drops them in <code>/data/</code> , adds TMDb API key to <code>.env</code> , runs <code>node server.js</code>
2 — First Launch	Server detects 12 films in both files (rewatches). 218 films enter ranking queue. TMDb fetches for all 630 unique titles are written to cache. First run takes a few minutes; all subsequent runs are instant.
3 — Taste Snapshot	Top genres: Drama (4.4), Thriller (4.2), Crime (4.1). Top keywords: slow-burn (4.7), psychological (4.5). Top directors: Kurosawa (4.9), Bergman (4.8), Park Chan-wook (4.7). Bayesian smoothing prevents a single 5-star Icelandic film from skewing the Iceland affinity.
4 — Ranking	218 films scored. Cure (1997) ranks 91%: Japanese cinema, slow-burn keywords, strong k-NN neighbours in Rashomon and Oldboy. The Conversation (1974) ranks 83%: 1970s era, paranoia keyword, Crime/Thriller. Transformers: Age of Extinction ranks 18%: low genre affinity, no keyword overlap, near-zero director affinity.

Step	What Happens
5 — Interaction	User filters to 'Only New Directors'. Finds Bong Joon-ho's Memories of Murder at #6 — model knows they gave Parasite 5 stars and the film's keywords/region match perfectly. User adds a new rating for Mulholland Drive — backend resolves metadata, runs triple duplicate check, updates taste model, re-ranks in real time.
6 — Export	User clicks Export. Browser downloads a Letterboxd v7-compliant CSV. User creates a new Letterboxd list, imports the file, and the Taste Matcher ranking is live on their profile.

5. Key Features

5.1 Multi-Dimensional Scoring Engine

Nine independent taste dimensions, each independently normalized and weighted, produce a composite affinity score. No single dimension dominates — low keyword affinity can be offset by high director and region affinity.

5.2 k-NN Nearest Neighbour Layer

A Jaccard similarity search over the entire rated history finds films most structurally similar to each watchlist candidate. Captures nuanced taste patterns that linear profile vectors miss — particularly for cross-genre and cross-regional mapping.

5.3 Bayesian-Smoothed Profiles

Low-sample profile entries (a director rated only once) are pulled toward the global mean. Prevents statistical outliers from distorting recommendations and makes the model robust even for users with smaller rating libraries.

5.4 Triple-Validation Duplicate Prevention

Manual entries are checked against three independent identifiers: Letterboxd URL, Title + Release Year, and TMDb ID. A film added via TMDb URL and re-attempted via title is correctly caught.

5.5 Zero-Downtime Hot Reload

The watchlist CSV can be swapped at any time. Clicking Reload Watchlist re-parses the new file, recomputes overlap, and rebuilds the ranked list without touching the ratings cache or taste model.

5.6 Letterboxd v7 Export Compliance

The export module generates a CSV that maps precisely to the Letterboxd v7 list import specification. Downloads directly in the browser; can be imported straight into any Letterboxd list.

5.7 Rewatch Ranking

Rewatches are scored by a formula that anchors to the user's own rating and uses the model as a secondary modifier, preventing the algorithm from contradicting explicit user judgment.

5.8 Local-First, No-Database Architecture

All state is persisted in flat JSON files. No database server to configure, no cloud dependency, no account required beyond a free TMDb API key. Fully portable.

5.9 User-Configurable Scoring Weights

The priority ranking system (introduced v3.1) lets users reorder the nine scoring dimensions via a drag-and-drop modal. Weights are computed via geometric decay and saved to `user_weights.json`, persisting across sessions.

5.10 Live Mutation System

Ratings can be added, films can be marked as watched, watchlist items can be removed, and rewatches can be hidden — all from the UI, all without restarting the server. Every mutation triggers an immediate taste model update and re-rank.

6. Unique Selling Points

6.1 Built for the Letterboxd Ecosystem

Not a generic recommendation engine retrofitted to accept CSV files. Designed from the ground up around Letterboxd's export formats, URL structure, and list import standard.

6.2 Mathematical Transparency

Every score is explicable. The UI displays score breakdown pills showing which dimensions contributed most to a film's ranking — in direct contrast to opaque streaming recommendation systems.

6.3 Purely Personal — No External Data Pollution

The taste model is derived exclusively from the user's own ratings. No collaborative filtering, no influence from other users, no advertiser weighting.

6.4 Operates on Your Own Curated Watchlist

Rather than suggesting films from a global catalogue, Taste Matcher ranks films the user has already decided are worth watching. It answers: among everything on your list, what should you watch first?

6.5 The k-NN Layer as a Differentiator

Most lightweight recommendation tools use only genre or tag matching. Taste Matcher's nearest-neighbour layer computes structural similarity across five dimensions simultaneously, identifying films that feel like your favourites even when genre labels are insufficient.

7. Current Drawbacks & Honest Limitations

Limitation	Impact	Mitigation
Synchronous API Fetching	First-run fetch for 500+ films can take several minutes	All subsequent runs are instant (cache-first). Sprint 3 backlog: Promise.all() batching with p-limit.
Linear Scoring Weights	Weights do not self-adapt to individual users. Default calibration suits most Letterboxd users but not edge-case taste patterns.	User priority ranking (v3.1) allows dimension reordering. Sprint 5 backlog: per-dimension numeric sliders.

Limitation	Impact	Mitigation
Cold Start Limitation	Users with fewer than ~30 ratings will see a flat, less personalised ranking — many dimensions fall back to the Bayesian global mean.	System is most powerful for active Letterboxd users with 100+ ratings. Bayesian smoothing prevents crashes on sparse data.
No Real-Time Sync	New Letterboxd ratings are not reflected until re-export and reload (or manual add-rating).	Manual add-rating endpoint partially mitigates for individual films. Long-term: Letterboxd OAuth if API opens.

8. Future Roadmap

8.1 Concurrent API Fetching with p-limit

Replace the sequential TMDb fetch loop with Promise.all() batching, regulated by a p-limit concurrency limiter. Projected to reduce first-run fetch time for a 500-film library from 8–12 minutes to under 90 seconds. Non-breaking; no cache migration required.

8.2 Per-Dimension Weight Fine-Tuning

Advanced mode allowing numeric slider control within a priority tier. For power users who want more granularity than rank order alone. Directly addresses the static-weights limitation.

8.3 Machine Learning Integration

Replace the static linear heuristic scoring function with a predictive ML model trained on the user's own rating dataset. Candidate architectures: Matrix Factorization and lightweight Neural Collaborative Filtering. The ML model would capture non-linear relationships between taste dimensions that the current linear weights cannot express. The system is already collecting the training data it needs with every rating added.

8.4 Letterboxd OAuth / Live Sync (Exploratory)

If Letterboxd opens a public OAuth API, integrate live sync so new ratings appear in Taste Matcher automatically. Eliminates the manual CSV refresh cycle and makes Taste Matcher a native Letterboxd companion.

9. Technical Stack Summary

Layer	Technology	Role
Backend Runtime	Node.js 18+	Application server and scoring engine
HTTP Framework	Express.js	REST API routing and static file serving
HTTP Client	Axios	TMDb API requests with error handling
CSV Parsing	csv-parse	Letterboxd CSV ingestion, header detection
External Data	TMDb REST API	Movie metadata (genres, credits, keywords)

Layer	Technology	Role
Frontend	Vanilla HTML / CSS / JS	Framework-free dynamic UI, zero bundle cost
Desktop Wrapper	Electron 28	Cross-platform native app with IPC bridge
State Persistence	Flat JSON files (/cache/)	No database — instant reads, portable
Environment Config	dotenv (.env)	API key and port management

10. Sprint 2 QA Sign-Off Summary

Taste Matcher v3.1 was validated through 86 manually executed functional test cases across six testing epics. All test cases passed.

Testing Epic	Tests	Status	Notes
System Boot & Cache Persistence	12	✓ PASS	JSON flat-file state recovery verified
Data Parsing & Overlap Detection	6	✓ PASS	Title+Year fallback confirmed working
Scoring Algorithm & Rewatch Math	14	✓ PASS	Bayesian smoothing and anchor metrics verified
UI Rendering & Filters	20	✓ PASS	DOM updates and lazy loading confirmed
Mutations & Export Compliance	20	✓ PASS	v7 Letterboxd format confirmed, browser download works
Edge Cases & Error Handling	14	✓ PASS	Graceful degradation on missing assets and bad input
TOTAL	86	✓ PASS	Ready for deployment

Closing Statement

Taste Matcher v2.1 represents a complete, production-ready refactor of the original prototype. It delivers a mathematically rigorous, locally-run, Letterboxd-native watchlist ranking engine that has no meaningful equivalent in the open-source landscape. The architecture is deliberately minimal — no database, no framework, no cloud — making it trivially deployable by any developer with a Node.js installation and a Letterboxd account.

The current version's limitations are not flaws to be hidden but engineering trade-offs documented and tracked. The Sprint 3 roadmap transforms each known constraint into a concrete, scoped improvement. The transition from a static linear model to a machine learning scoring engine is not aspirational — it is the natural next step for a system that is already collecting the training data it needs with every rating the user adds.

Taste Matcher exists because the problem it solves is real, the solution is non-trivial, and the existing tools do not address it. That combination is the foundation of a project worth building.